Searching for a Self-Contained Non-Neural MNIST Classifier with LLM Agents

Report on the DSA5105 after-class experiment (lecture slides 65–67)

Kushagra Bhatnagar · 9 October 2026 · Code: github.com/kushagrab21/non-neural-mnist-search

1 Problem statement

The experiment on slide 66 restricts the classifier at the moment it is used, not while it is being developed. These restrictions are stated first, because they decide what kind of program can succeed.

Input and output:

At inference the program receives a single 28×28 greyscale image, that is, 784 integer intensities between 0 and 255, and must return one digit from 0 to 9. It may not read training images, query a database or load an external parameter file, and it may not contain a neural network.

Consequence:

Development, by contrast, is unrestricted: the 60,000 MNIST training images may be used to fit as many parameters as desired. Every fitted quantity, however, has to be written into the program's own source code as a constant. The program is therefore a compressed summary of the training set, and every piece of knowledge it uses has a cost in bytes. The relevant measure is thus accuracy obtained per byte of program, and not accuracy alone. Slide 65 then asks whether an automated search over such programs can move this trade-off far enough to change the method one would choose over a small neural network.

Experimental boundary:

Three things were fixed before the search began.

1. The search space: a single self-contained Python file, `classifier.py`, using only NumPy.
2. The data available for fitting: 50,000 of the 60,000 training images.
3. The scoring rule: accuracy, exact file size and per-image latency, measured on the remaining 10,000 training images (on which no program was ever fitted) alongside a small convolutional network (CNN) on the same machine.

The official MNIST test set of 10,000 images was kept out of the search entirely and opened only after the final program had been frozen.

Slide 67: Did search improve one method, or change the method you preferred?

Over three iterations the search raised the development accuracy of a non-neural program from **84.89%** to **97.83%**. On the MNIST test set the selected program classifies **9,809** of 10,000 images correctly (**98.09%**), against 9,883 (98.83%) for a small CNN. It is about **9.6 times smaller** (27,654 bytes against 265,390) and was faster per image in an earlier development measurement (0.30 ms against 0.50 ms).

Before the experiment I would have chosen the CNN without hesitation. After it, the choice depends on the constraints: where model size or latency matters, I would choose the searched program.

2 The generating workflow

The program was produced by a loop in which three separate sessions of one language model (Claude Opus 5.5) hold different responsibilities, while ordinary code does the fitting and the measuring. The separation has one purpose: the session that decides what to try next never fits or scores a program itself, so the accuracy, size and latency on which it bases its decisions are measured by code it cannot change. It also read figures that the build scripts reported about themselves, such as scores on their internal hold-out images; these are identified as such in Section 3.

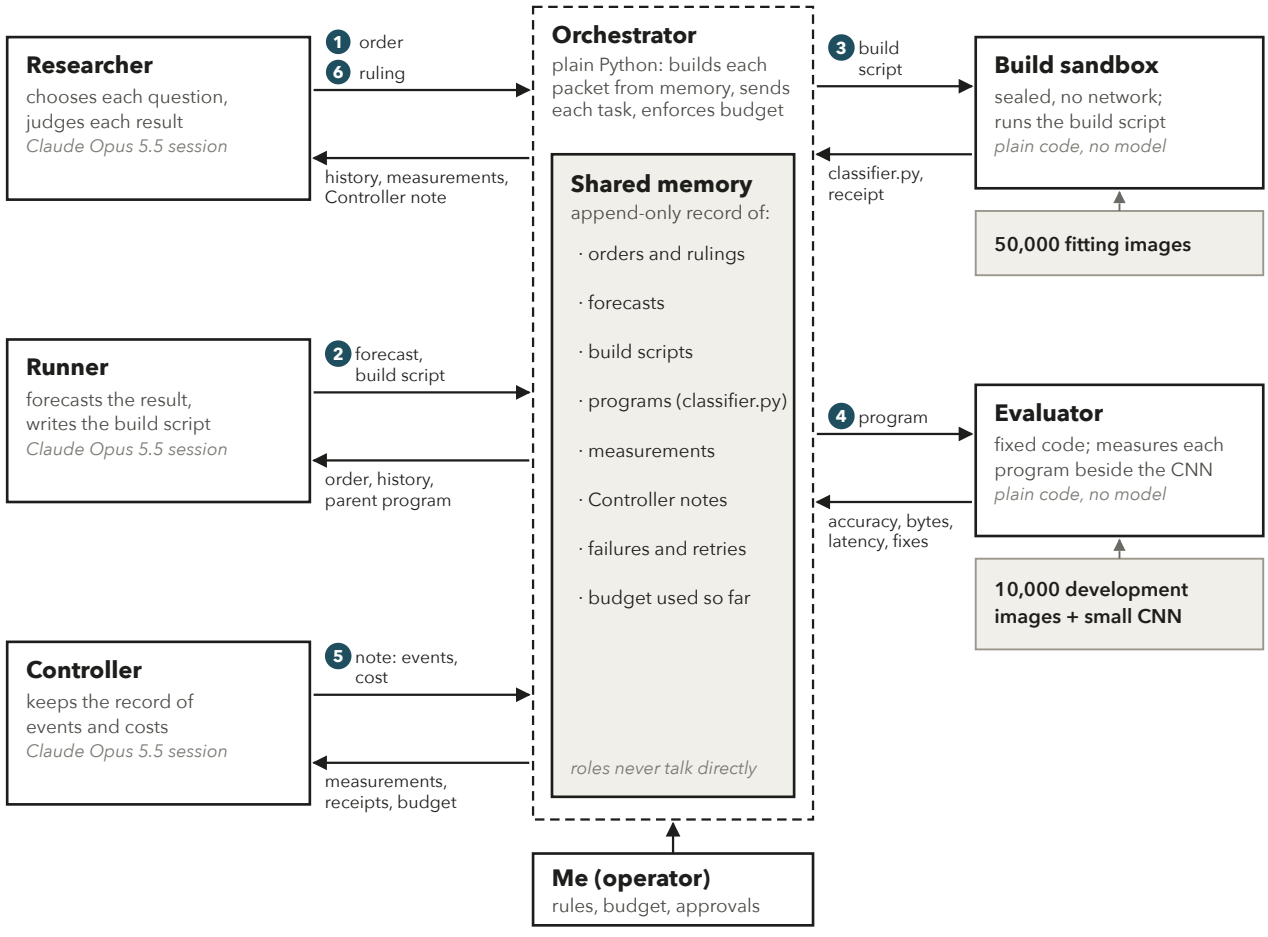


Figure 1: Architecture of the generating workflow. Three model sessions (left) and two pieces of plain code (right) exchange information only through a shared, append-only memory kept by the orchestrator. Arrows show what each component reads and writes; numbered arrows give the order of one iteration, matching Figure 2.

The components in Figure 1 have the following roles.

Researcher (a model session): chooses the question for each iteration, writes the order, and later judges the result.

Runner (a model session): forecasts the result of the order and writes the build script that implements it.

Controller (a model session): writes a note of what happened in each iteration and what it cost.

Build sandbox (plain code): runs the build script in a sealed container with no network access that holds only the 50,000 fitting images.

Evaluator (plain code): measures each emitted program on the 10,000 development images beside the CNN.

Orchestrator and shared memory (plain code): before each task, the orchestrator assembles a packet from the memory (the order, the history of results and rulings, failures and the budget remaining) and sends it to the right component; every reply is appended to the memory. The three sessions never message each other directly.

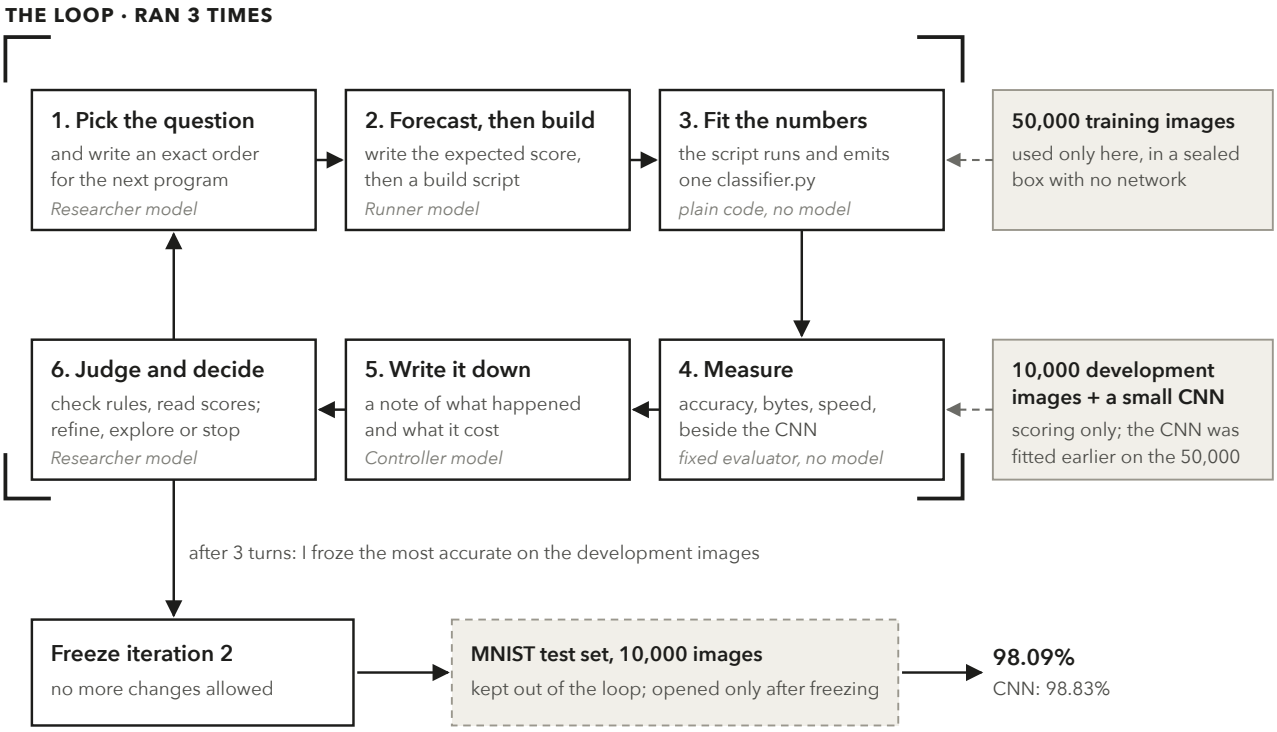


Figure 2: The same workflow as a loop over time, with the data each step may use. The loop ran three times; the test set lies outside it and was opened only after the selected program had been frozen.

One iteration proceeds as follows (the numbers match both figures).

1. The Researcher states a question and writes an exact order for one candidate program: the features, the fitting procedure and a size limit.
2. The Runner first records a forecast of the result and then writes a build script that implements the order.

3. The build sandbox runs the script, which estimates the numerical parameters and emits a single self-contained `classifier.py`. Within the 50,000 images the script sets aside 5,000 of its own to choose the regularisation strength.
4. The evaluator, which no model can modify, loads the emitted file and measures its accuracy on the 10,000 development images, its exact size in bytes and its latency per image, with the CNN measured under the same conditions.
5. The Controller records what happened and what it cost. This record supports the budget accounting and is not needed for the measurements themselves.
6. The Researcher reads the measurements, checks that the program obeys the rules of slide 66 (no stored examples, no external files, no neural network), and decides whether to refine the current direction, explore a different one, or stop.

After the loop I selected the candidate with the highest development accuracy (iteration 2) and froze it; only then was the test set opened.

Search budget

Search run (three iterations)	Recorded use
Model tasks (calls to the three sessions)	18, including 3 retries
Settled work time / time from start to close	19.4 min / 58.7 min
Evaluation requests / isolated worker processes	216,744 / 132
Model usage valued at list price (the run used a flat subscription, so no extra charge was incurred)	$23.82
Limits fixed in advance	2 hours, 6 candidates, 42 tasks
Ceiling on model usage at list price	$15, raised to $30 for iteration 3

Settled work time adds up the periods in which a model call, a build or an evaluation was running; the start-to-close time also includes the gaps between them.

An evaluation request is one image sent to a program for classification, including warm-up calls, for both the candidate and the CNN; each comparison ran in fresh, isolated worker processes.

Not included: three earlier pilot runs of the same setup (11 further model tasks), which produced the starting program, and my own time spent designing the rules, the evaluator and the data split, which was not metered.

The CNN was prepared before the search as the fixed neural comparator; its training record reports six epochs in 11.8 s, which was not independently reproduced, and the effort of designing it is unmeasured.

What went wrong while running it

The run was not fully unattended.

1. In iteration 2 the Runner's forecast call failed after an authentication lapse and then repeatedly returned output in the wrong format. The run stopped, and I raised the retry limit from one to three per task before continuing.
2. The Controller's record for iteration 2 exceeded its cost limit twice ($1.74 against $1.00, then $3.55 against $3.50 after the per-role limits were rebalanced within the same $15 total). That record was never accepted, and the Researcher judged iteration 2 on the evaluator's measurements alone.
3. The run halted after iteration 2 because the cost allowance was exhausted. Iteration 3 ran only after I authorised exactly one further candidate, for which the list-price ceiling was raised from $15 to $30 (paid extras remained disabled); the recorded usage of $23.82 therefore exceeds the original ceiling.

None of these interventions repeated or altered a measurement.

3 The three iterations

The starting point was a program from my pilot runs: a linear ridge classifier on 2×2-averaged pixels, scoring 84.89% on the development images at 3,905 bytes. Each card below records the question the Researcher asked, the change it ordered, the forecast written before the build, the measured result, and the decision that followed. Figure 3 places the results on the accuracy–size plane.

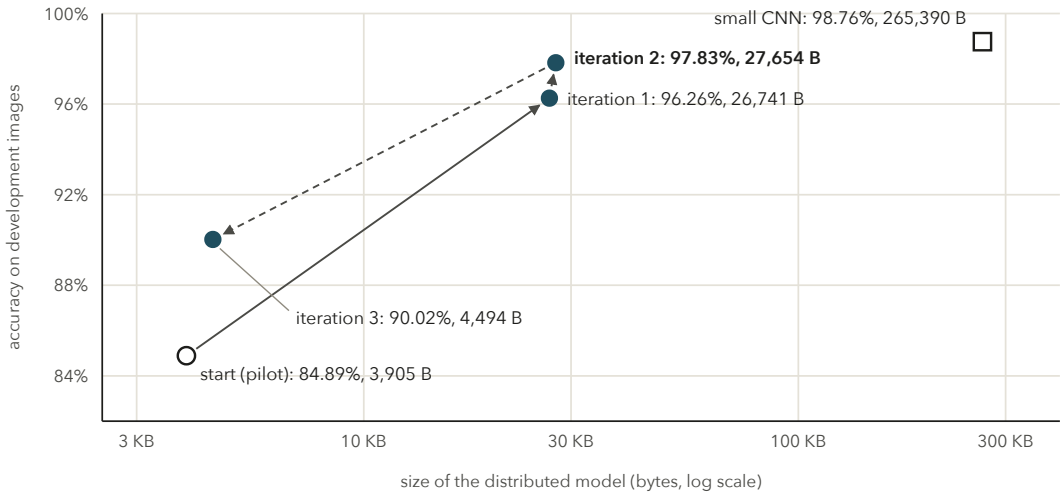


Figure 3: Development accuracy against the size of the distributed model (log scale). Solid arrows: refinements; dashed arrow: the exploration towards a smaller program. The CNN is shown for reference and was not part of the search.

Iteration 1 — adding interactions between features

direction: refine the starting point

QUESTION	How much accuracy does a classifier that can use products of features gain over the linear starting point, and at what cost in bytes?
CHANGE	Average 2×2 blocks (196 values); project onto 40 principal directions learned from the fitting images and rescale them; append all 820 pairwise products of the 40 values; fit a ridge classifier on the resulting 860 features; store the weights as 8-bit integers.
FORECAST	96.5–97.8%; 24–30 KB; 0.28–0.40 ms per image.
MEASURED	**96.26%**; 26,741 B; 0.247 ms. CNN, re-measured in the same comparison: 98.76%; 0.512 ms.
DECISION	Refine. On the build script's internal hold-out, accuracy was nearly flat across the smaller regularisation strengths (a figure reported by the script), which the Researcher read as the model being limited by its input representation rather than by over-fitting; it proposed normalising the input (deslanting) as the cheapest next step. It also noted that latency was dominated by the fixed cost of calling the program, so that bytes, not time, were the cost that mattered.

Iteration 2 — deslanting the input

direction: refine iteration 1

QUESTION	With everything else unchanged, does deslanting each image before the same pipeline raise accuracy above 96.26%?
CHANGE	Before averaging, the program computes the image's centre of ink and its slant from second moments, then shears and recentres the image by bilinear interpolation. This step stores no learned values. The projection and weights were refitted on deslanted images.
FORECAST	97.0–97.8%; 120–220 errors fixed and 40–90 introduced relative to iteration 1; 27.7–28.5 KB; 0.26–0.32 ms.
MEASURED	**97.83%**; 27,654 B (+913 B); 0.299 ms. On the same images as iteration 1: 209 errors fixed, 52 introduced.
DECISION	Explore next. Deslanting was taken as validated; since it costs about 1 KB, the Researcher chose to reuse it in a much smaller linear program as the next candidate, to fill the gap between 3.9 KB and 27 KB. A further refinement of iteration 2 (56 principal directions, about 52 KB) was planned to follow, but was never run.

Iteration 3 — how small can the program be?

direction: explore

QUESTION	Below about 6.5 KB, how much accuracy does deslanting add to a plain linear classifier?
CHANGE	Deslant, average 2×2, then a linear ridge classifier on the 196 values with 8-bit weights. An optional extension using row and column ink profiles was tested inside the build and rejected (+0.28 points, below its 0.5-point threshold).
FORECAST	88.5–91.5%; 4.4–5.2 KB; deslanting worth 3–5 points on the build's internal check.
MEASURED	**90.02%**; 4,494 B; 0.283 ms.
DECISION	Stop. This decision was mine rather than the Researcher's: only this one further candidate had been authorised.

I selected iteration 2 as the submitted classifier because it had the highest development accuracy, and froze it. Iteration 3 is retained as the smallest point found but was not evaluated on the test set. The build scripts also ran an internal check, on their own 5,000 held-out fitting images, of the same model without deslanting: deslanting was worth 1.30 points to the quadratic model of iteration 2 and 6.02 points to the linear model of iteration 3.

What this section does not show

The development images guided every decision, so the accuracies above are selection-biased and come from a single split, measured once each. The two internal deslanting checks were reported by the build scripts and were not reproduced by the evaluator. Iteration 3 is a different model rather than iteration 2 with one part removed, so it does not measure the value of any single component of iteration 2.

4 The classifier obtained

The submitted classifier is iteration 2: a single Python file of 27,654 bytes that imports only NumPy and the standard `base64` module, which decodes its embedded constants. No language model takes part when it classifies an image; the role of the models ended when the file was written. Figure 4 shows its architecture.

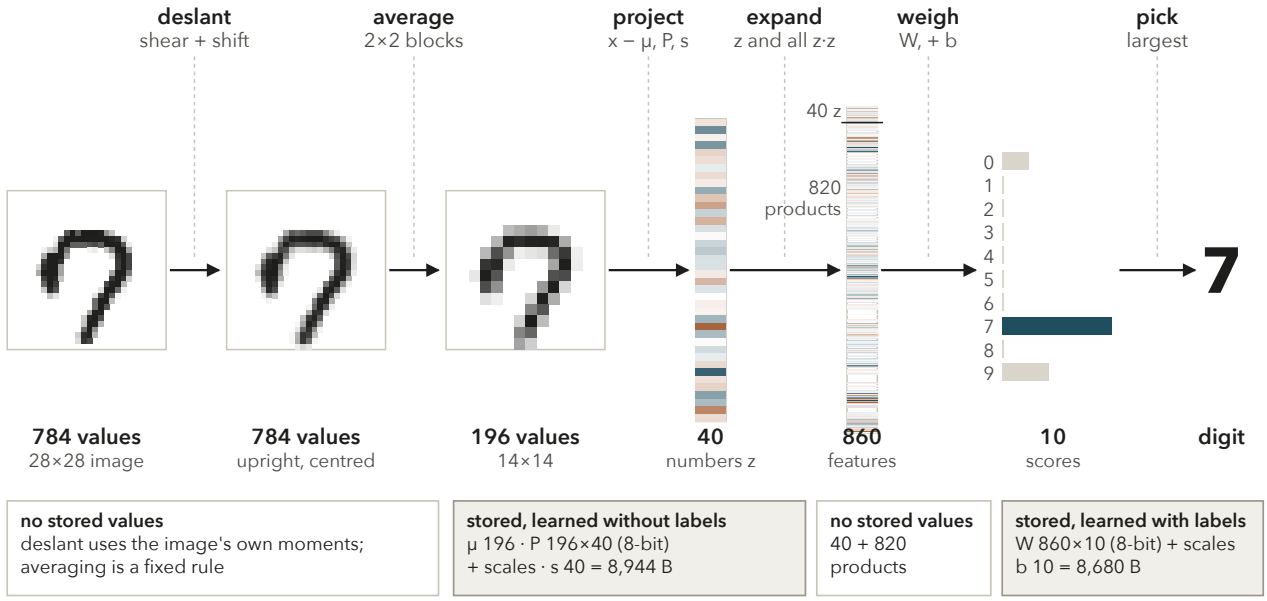


Figure 4: Architecture of the submitted classifier, with one development image (the 7 that also appears in Figure 8) followed through every layer. The coloured columns are the actual 40 numbers z and 860 features computed for this image (brown positive, blue negative); the bars are its ten scores. The boxes along the bottom show which steps use stored, learned values and how many bytes they occupy.

The learned values were estimated from the fitting images only: the mean image and the 40 principal directions from 45,000 of them, the regularisation strength on the other 5,000, and the final classifier weights on all 50,000. Storing the projection and the weights as 8-bit integers, each with a scale factor, is what keeps the file small. Table 1 accounts for every byte.

Table 1: Contents of `classifier.py`	Bytes
Projection, 196 × 40 values at 1 byte	7,840
Classifier weights, 860 × 10 values at 1 byte	8,600
Mean image, scale factors and biases (32-bit floats)	1,184
Learned values	**17,624**
The same values written as base64 text in the source	23,516
Program code and remaining text	4,138
File total	**27,654**

Counted directly from the file. Base64 encoding adds about a third to the stored values; at run time they are decoded to 32-bit floats, so the memory used while running is larger than the file.

Both principal ideas of the program are established techniques. LeCun et al. (1998) report a classifier that projects digits onto 40 principal components and applies a second-degree polynomial (3.3% test error), and they describe deslanting digits by second moments; in that paper the polynomial classifier is reported on images that were not deslanted. What the search assembled is the combination of the two, together with the averaging step and the 8-bit storage. Section 7 explains why this structure works.

5 Results on the test set

Both programs were frozen before the test set was opened. Each was run once over the same 10,000 official MNIST test images, whose images and labels were checked against recorded hashes. The searched program was tested first; the CNN, whose weights had been fixed before that, was tested afterwards at my request and without retraining. No test result was used to change either program. The test archive was therefore opened twice, once for each program.

Table 2: Frozen programs on the MNIST test set	Searched program	Small CNN
Test accuracy	**98.09%**	**98.83%**
Correct / errors (of 10,000)	9,809 / 191	9,883 / 117
Development accuracy, for reference	97.83%	98.76%
Size of the distributed model	27,654 B	265,390 B
Latency per image (development measurement)*	0.299 ms	0.497 ms
Model	non-neural, 17,624 B of values	LeNet-style, 65,162 parameters
Runtime dependencies	Python, NumPy	Python, NumPy, PyTorch

CNN size: 527 B architecture code, 566 B image adapter and 264,297 B of weights. Neither size includes Python or the libraries. *Latency was measured earlier, on the development images and not during the test pass, as the median of three blocks of 128 images after 50 warm-up calls (ranges 0.294–0.339 ms and 0.494–0.504 ms). The CNN was re-measured alongside each iteration (0.494–0.512 ms); Table 2 uses the comparison run with iteration 2. It includes sending each image to the program's process and receiving the answer, and excludes start-up.

The two programs agree on most images: both are correct on 9,747 and both are wrong on 55. The CNN alone is correct on 136, and the searched program alone on 62. The CNN's advantage is concentrated in two digits (Figure 5). Of its 74 fewer errors, 25 come from the digit 9 and 19 from the digit 2. On the digits 1 and 5 the searched program makes fewer errors than the CNN. The searched program's test accuracy is 0.26 points above its development accuracy (the CNN's is 0.07 points above), so there is no sign that selection on the development images inflated its estimate, although a single comparison of this kind cannot rule it out.

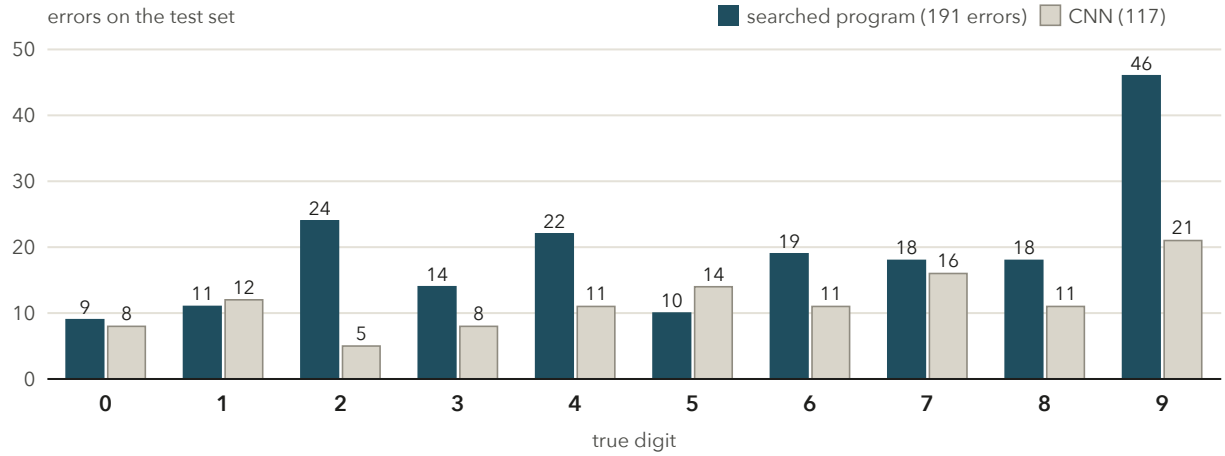


Figure 5: Errors on the 10,000 test images by true digit. Counts from the two frozen programs' test predictions.

What this section does not show

The accuracy gap is not a sampling artefact: of the 198 images on which exactly one program is correct, the CNN is correct on 136 (two-sided sign test, $p < 10^{-6}$). The comparison is nevertheless between a searched non-neural method and a standard small CNN that received no search of its own, on one test cohort. The latency figures come from the development measurement, include communication overhead, and are not a deployment benchmark. Performance on other handwriting (for example QMNIST) was not assessed.

6 Inferences

6.1 Search substantially improved one method

In three iterations, 19.4 minutes of settled work (model calls, builds and evaluations) and an estimated $24 of model usage, the development accuracy of the non-neural program rose from 84.89% to 97.83%, and the frozen program reached 98.09% on the test set. The largest step was the first (+11.37 points), which introduced the principal-direction projection and the pairwise products together. The search run did not separate the two. The follow-up analysis in Section 7 shows that, on the deslanted images of iteration 2, the products alone are worth 8.47 points over a linear head on the same 40 coordinates; this is not a split of the iteration 1 step, which had no deslanting. Deslanting added a further 1.57 points for 913 bytes.

6.2 It did not overturn the CNN on accuracy

The CNN still makes 74 fewer errors on the test set, and the difference is concentrated in the digits 9 and 2. On accuracy alone, the neural model remains the better choice.

6.3 It changed the preferred method under constraints

When the size of the shipped model or the time per image matters, the searched program gives up 0.74 points of accuracy in exchange for a model 9.6 times smaller, about 40% lower latency in the development measurement, and no dependency on a deep-learning library. The latency advantage should be read with care: much of each figure is the fixed cost of calling a separate process, and the difference in computation alone was not measured. Before the experiment I would not have considered a non-neural program a serious alternative. After it, the choice depends on these constraints. In the terms of slide 67, search improved one method and, for part of the range of requirements, also changed the method I prefer. It did not move between method families: every iteration is a ridge classifier on constructed features, so the change of preference is between this family, once searched, and the CNN.

6.4 The search assembled known techniques quickly

Every step the Researcher chose (principal components, pairwise products, deslanting, ridge regression) is a classical technique, and LeCun et al. (1998) already report the first two together. What the search contributed was the speed and low cost with which it assembled them, combined them and fitted them under a byte constraint. This bears on the point of slide 65: the design effort that a non-neural method needed in order to come within 0.74 points of a small CNN was, in this case, under an hour of machine search with my supervision, on top of the unmetered preparation.

6.5 Each step was a stated hypothesis that the measurement then tested

The forecasts written before each build were close to the measurements, though not exact. The accuracy of iteration 3 (90.02%) fell inside its forecast range; that of iteration 2 (97.83%) fell just above its upper end of 97.8%; and that of iteration 1 (96.26%) fell 0.24 points below its range. Among the other forecasts, iteration 1 ran faster than predicted (0.247 ms against 0.28–0.40 ms), the iteration 2 file came in 46 bytes below its forecast size range, and the deslanting gain in iteration 3 (6.02 points) exceeded the forecast 3–5 points. The decisions also followed reasons that the next measurement checked: the Researcher predicted that deslanting would fix more errors than it introduced, and it fixed four times as many (209 against 52). The internal checks reported by the build scripts further suggest that deslanting matters more for a weaker model (+6.02 points for the linear model against +1.30 for the quadratic one), which is consistent with a quadratic model being able to absorb part of the slant variation that a linear model cannot.

What these inferences do not establish

The experiment consists of a single search run with one development split and one test cohort. It was not compared with a single model searching alone, with a simple grid search, or with a human designing the same program, so it does not show that this three-role loop is better than a simpler search. The pilots and the design of the rules were not metered, so $23.82 and 19.4 minutes are not the full cost, and the cost ceiling was raised once, from $15 to $30, to allow iteration 3. The CNN was not searched, and a searched neural model would probably widen the accuracy gap.

7 Dissection of the classifier

This section takes the submitted program apart by asking the questions a reader would ask of its code, and answering each from first principles. "Works well" is meant in a specific sense: the program reaches 98.09% on the test set with 17,624 bytes of learned values and a fit that needs no gradient descent, while still making 74 more errors than a small CNN. The answers explain that trade, not raw accuracy.

Several answers rest on a follow-up analysis run after the search. It refitted five variants of the program on the same 50,000 fitting images, using the same recipe and the same rule for choosing the regularisation strength, and compared them with the frozen program on the 10,000 development images. Intervals quoted below are descriptive: they come from 2,000 paired bootstrap resamples of those images. The rule chose different strengths for different variants, and for three of them (including the submitted 40-coordinate form) it chose the smallest value on the grid, so the variants are comparisons within this recipe rather than best possible versions. The test set was not used again. Before any comparison, the analysis reproduced every stored value of the frozen program bit for bit and recomputed its scores to within 3×10^{-6}, with no changed prediction. Each answer carries a tag: FROM THE CODE for what the program does by construction, MEASURED for results of this analysis or of the search, and REASONED for explanation that no measurement here tests.

Table 3: Variants of the program on the development images	Features in head	Accuracy	Errors	Learned values
20 coordinates, quadratic head	230	95.94%	406	14,104 B
40 coordinates, quadratic head	860	97.82%	218	43,384 B
60 coordinates, quadratic head	1,890	97.92%	208	88,664 B
40 coordinates, linear head	40	89.35%	1,065	10,584 B
40 coordinates, all values as 32-bit floats	860	97.82%	218	66,744 B
Submitted program (8-bit projection and head)	**860**	**97.83%**	**217**	**17,624 B**

The first four variants store the projection in 8 bits and the head in 32-bit floats, so that the number of coordinates and the type of head are compared without changing how the head is stored. Learned values exclude the base64 text and program code that bring the submitted file to 27,654 bytes.

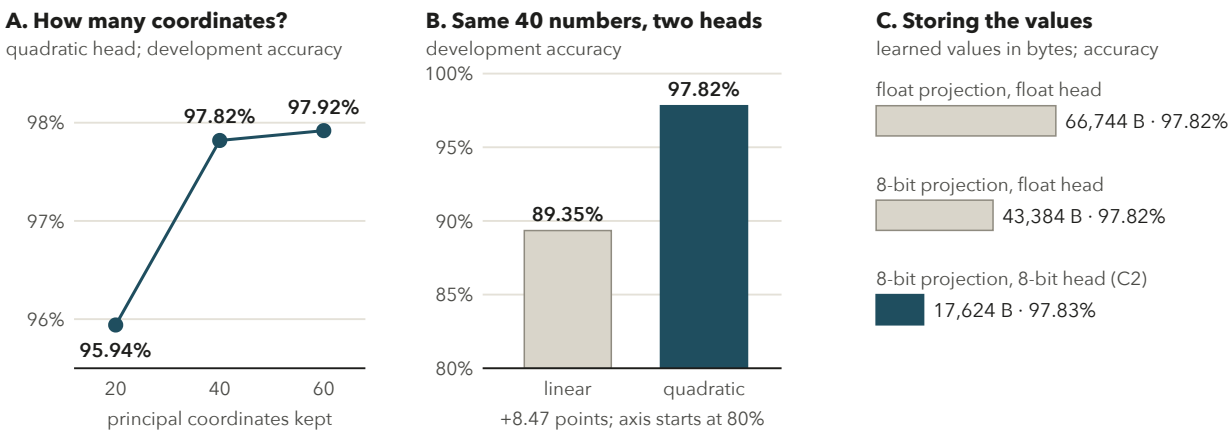


Figure 6: The three comparisons behind Questions 3, 5 and 7, on the development images. A: fewer coordinates cost accuracy, more add little. B: with the 40 coordinates held fixed, the quadratic head adds 8.47 points. C: 8-bit storage removes three quarters of the bytes without changing accuracy.

Q1 What does the program compute, written as a single formula?

FROM THE CODE Writing a for the image scaled to $[0, 1]$, the program computes

```
x   = average_2x2( deslant(a) )            196 numbers
z   = s ⊙ ( P^T (x − μ) )                    40 numbers
φ   = [ z_1,…,z_40,  z_i·z_j for all i ≤ j ]   40 + 820 = 860 numbers
digit = argmax over d of ( w_d · φ + b_d )
```

Every class score is therefore a quadratic function of 40 numbers, and those 40 numbers are a linear summary of a normalised 14×14 image. Here μ is the mean image, P the 196×40 projection (stored as 8-bit integers with one decoding scale per column), s the 40 rescaling factors that give each coordinate roughly unit variance, and W and b the class weights and biases (W also stored in 8 bits with one scale per class). Only these are learned (Figure 4, Table 1); deslanting, averaging, the products and the final choice are fixed rules. The mean μ and the directions P were estimated without labels; only the weights W and biases b used the digit labels.

Q2 Why is the image deslanted and averaged before anything is learned?

FROM THE CODE Deslanting computes, for each image, the centre of its ink and the slope of its main axis from second moments, then shears and shifts the image so that the axis is upright and centred. Averaging 2×2 blocks then turns 784 pixels into 196 values. Neither step stores anything.

REASONED Slant and position vary from writer to writer but say nothing about which digit was written. If they are left in the image, the learned directions must spend part of their capacity describing them. Removing them by rule costs no learned bytes, and averaging quarters the size of the projection matrix while discarding only pixel-level detail.

MEASURED Three numbers describe deslanting, and they measure different things. Between iterations 1 and 2 the evaluator measured $+1.57$ points on the development images (209 errors fixed, 52 introduced) for 913 bytes of code. The build scripts' own checks on their internal hold-out, comparing the same model with and without deslanting, gave $+1.30$ points for the quadratic model and $+6.02$ for the linear model of iteration 3. The contribution of averaging alone was not measured.

REASONED The larger gain for the linear model suggests that deslanting helps most where the model is least able to absorb slant itself. The two models differ in more than their head, however, and those internal checks were not reproduced by the evaluator, so this remains an interpretation.

Q3 Why can 196 values be replaced by 40 numbers?

REASONED Neighbouring pixels in a digit are strongly correlated, and the ways in which one handwritten digit differs from another (width, loop size, stroke position) are few. The images therefore lie close to a low-dimensional subspace, and principal component analysis finds the directions of that subspace that carry the most variation.

MEASURED On the 45,000 images used to fit them, 20 directions account for 82.3% of the variation of the deslanted, averaged images, 40 for 93.0% and 60 for 97.1%. For classification, cutting from 40 to 20 coordinates lost 1.88 points (221 correct answers broken, 33 errors corrected), with losses in every digit. Going from 40 to 60 gained 0.10 points (54 corrected, 44 broken; descriptive 95% interval -0.09 to $+0.29$), while the head grew from 860 to 1,890 features and the stored values roughly doubled. In this recipe, then, 40 is near the point where further coordinates stop paying for themselves, although the analysis does not show that 40 is optimal.

Q4 What do the learned directions look like?

FROM THE CODE Each direction is a column of P, that is, a 14×14 pattern of weights. Projecting an image onto it measures how strongly the image resembles that pattern relative to the mean. Figure 7 draws the mean and the first nine directions directly from the values stored in the file.

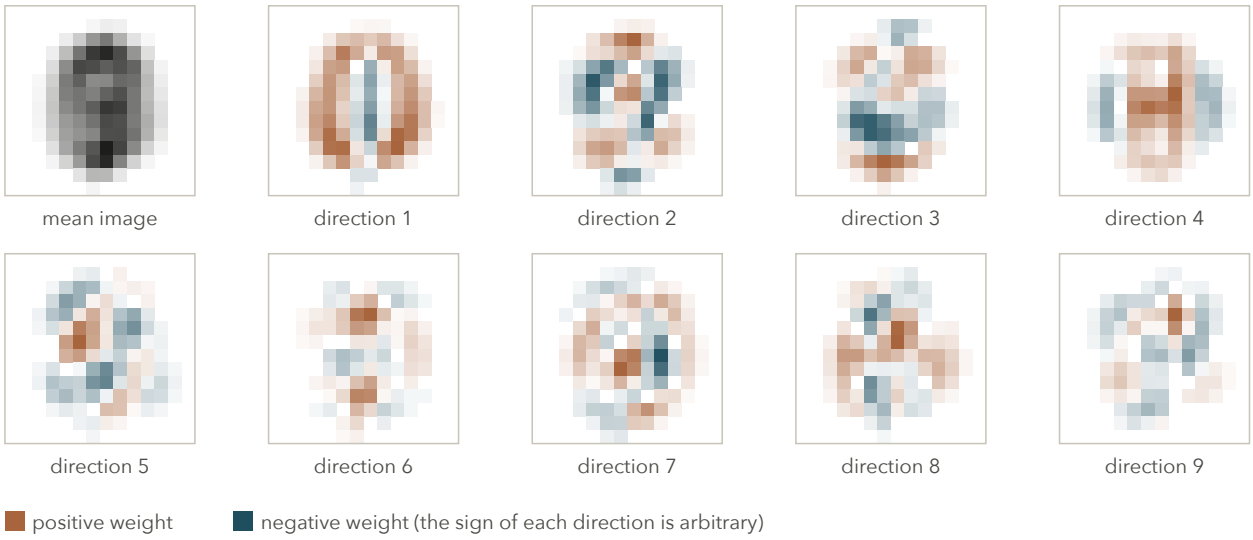


Figure 7: The mean image and the first nine of the 40 stored directions, decoded from `classifier.py`. Each is a 14 × 14 pattern on the deslanted, averaged image.

REASONED The mean is an upright blur, as deslanting would produce. The first direction weighs a ring against a central vertical stroke, so a single number already separates 0-like from 1-like shapes. The next few directions contrast different regions of the digit, for example its upper and lower parts. Later directions become finer patterns that no longer resemble whole strokes. This is a description of the pictures; the analysis did not test which direction encodes which visual property, and individual coordinates should not be read as named stroke detectors.

Q5 Why multiply the 40 numbers together?

REASONED A linear score gives each coordinate a fixed weight and adds the results, so the boundary between two digits is a flat hyperplane in the 40-dimensional space. A product such as $z_2 \cdot z_4$ lets the effect of one coordinate depend on the value of another, and a square such as z_2^2 responds to large values on either side of zero. With all 820 such terms the boundaries become curved surfaces, which can follow the shape of a digit class rather than cut straight through it. The terms are computed from the current image; nothing about individual training images is stored.

MEASURED With the same 40 coordinates, a linear head reaches 89.35% and the quadratic head 97.82%: 868 errors corrected and 21 introduced, a gain of 8.47 points (descriptive 95% interval 7.94 to 9.02), with a net gain in all ten digits. This is the largest effect among the refitted variants. The fitted program also depends on these terms jointly with its linear weights: deleting the quadratic terms from the submitted program, without refitting, drops it to 43.86%. That figure shows reliance within this fit, not what a linear model could learn.

Q6 What does a single decision look like?

MEASURED The score difference between the chosen digit and its strongest rival splits exactly into four parts: the bias, the 40 linear terms, the 40 squares and the 780 cross-products. Figure 8 shows this for four development images chosen by a rule fixed in advance: for each kind of case, the image whose margin is the median among all images of that kind. The "linear model" in Figure 8 is the separately fitted linear head of Question 5.

In the second row a 7, which the linear model calls a 9, is classified correctly. The bias and linear terms together favour 9 (−0.147), and the squares and cross-products reverse the decision (+0.567), so that the 7 wins by 0.420. The largest single term is $z_2 \cdot z_4$ (+0.282). In the first row, an ordinary 0, about four fifths of the margin (0.673 of 0.830) comes from cross-products. The last two rows are failures. In the third row the separately fitted linear model reads a 7 correctly, but this program calls it a 3; here its own bias and linear terms already favour the 3 (+0.078), so the error is not caused by the quadratic terms alone. In the fourth row a 4 with a nearly closed top is called a 9; there, the bias for 9 outweighs a negative contribution from the squares.

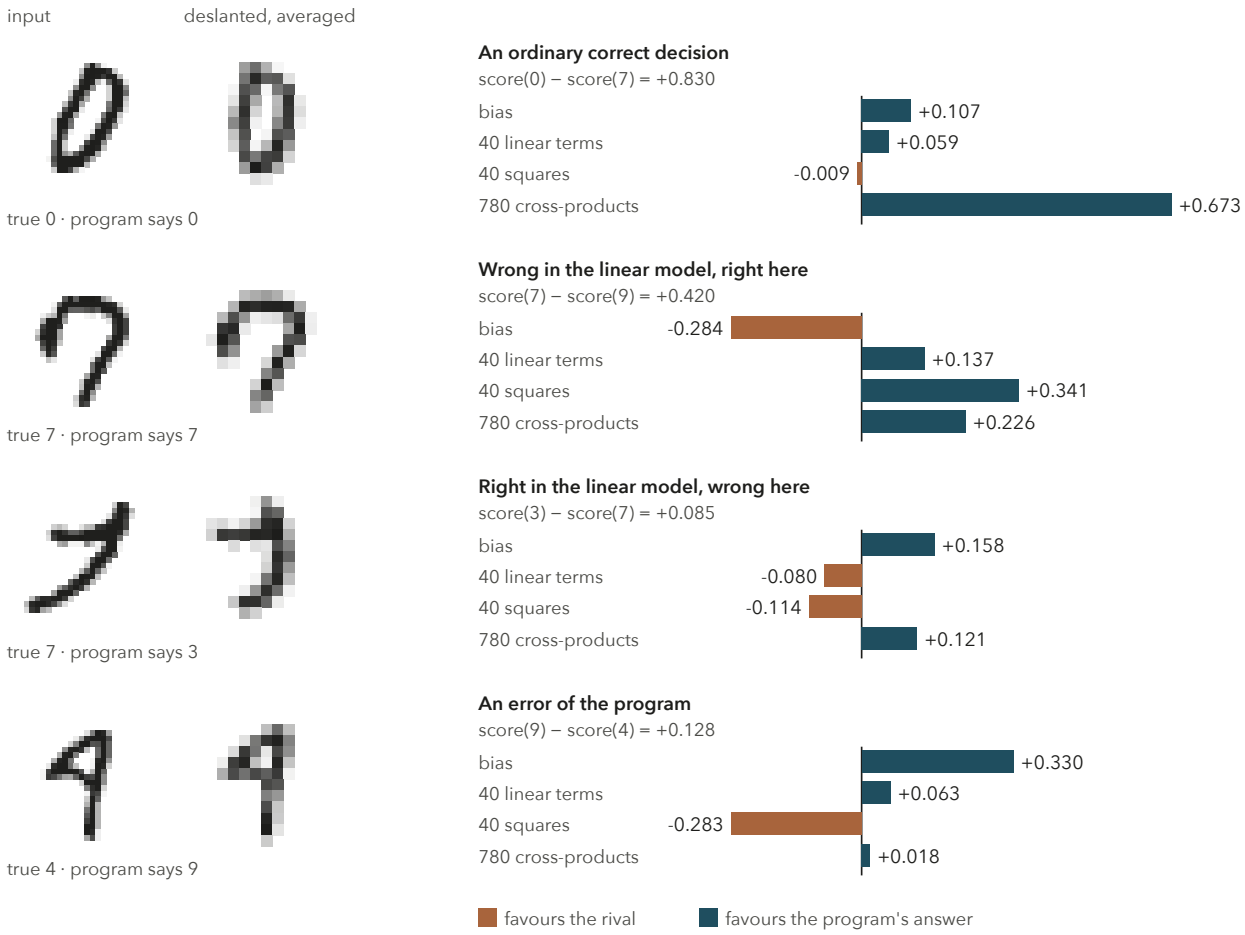


Figure 8: Four decisions on development images, from pixels to score difference. Bars show how much each group of terms contributes to the difference between the program's answer and its strongest rival.

Q7 Why are 8-bit values enough?

REASONED The output depends only on which of ten scores is largest, not on their exact values. Rounding each weight to one of 256 levels perturbs every score slightly, and that changes the answer only for images whose two best scores are already nearly tied.

MEASURED Storing the projection in 8 bits instead of 32-bit floats, with the head refitted, gave identical predictions on all 10,000 development images. Storing the head in 8 bits then changed 11 predictions: 4 errors corrected, 3 correct answers broken, and 4 wrong answers changed to other wrong answers. The learned values shrink from 66,744 to 17,624 bytes (−73.6%) at no measurable cost in accuracy. At run time the values are decoded back to 32-bit floats.

Q8 So what has the program absorbed, and where does it stop?

REASONED The program handles three kinds of regularity in handwritten digits, each in a different place. Variation in slant and position is removed by a fixed rule and costs no learned bytes. Shared variation in shape is captured by the mean and 40 directions, learned without labels (8,944 of the 17,624 bytes). Which combinations of those shape variations belong to which digit is captured by the 860×10 weights, learned with labels (the remaining 8,680 bytes). A digit is thus recognised less by any single feature than by which shape variations occur together, and 8-bit storage keeps all of this at about a quarter of the size of 32-bit floats.

The limit is the other side of the same design. Every decision is a quadratic function of a 40-number summary of a 14×14 image. Local details that a summary of global shape can blur, such as whether the top of a 4 is closed, may explain why the CNN's advantage concentrates in a few digits (9 and 2 in Figure 5; the program's own errors are highest for 9, then 2 and 4), and the CNN's local filters plausibly capture them better. This explanation is consistent with Figure 8 but was not tested here.

What this section does not show

The comparisons use the same 10,000 development images that guided the search, so they explain the program on that cohort rather than estimate its accuracy elsewhere; the test result in Section 5 is unchanged. The intervals describe variation over images for fixed fitted models, not over refits. Changing the number of coordinates also changes the number of products and the selected regularisation, so the variants are comparisons within one recipe, not separate causes. The analysis did not test a random projection in place of the principal directions, the contribution of averaging, or what individual directions encode.

References

LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

Liu, Y. (2026). *LLMs as Optimizers*, DSA5105 lecture slides 65–67, National University of Singapore.

Code: the submitted classifier, the three iteration programs and the build scripts that produced them, the CNN baseline, the workflow code and the Section 7 analysis are at https://github.com/kushagrab21/non-neural-mnist-search.